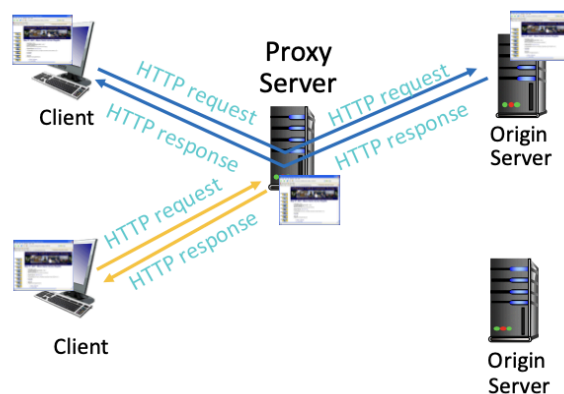


Step 4: Proxy server Implementation

Proxy Server

What is the difference between request handling between a web server and a proxy server?

The main difference is that the proxy server acts as both a client and server. It accepts client requests as a server and then makes requests towards the origin server to obtain the file (hence acting as the client in the second case.). We used the model below to implement our proxy server. **Find the test method of the proxy_server.py at the end of this section.**



The proxy server acts as an intermediary between the client and the origin web server. The client does not know the address or port of the origin server and sends all HTTP requests directly to the proxy server. The origin server's address is predefined in the proxy program as localhost on port 9090.

For example, the client requests test.html using:

```
curl -i http://localhost:8888/test.html
```

The client only specifies the proxy server's port, 8888. The proxy is responsible for communicating with the origin server on port 9090.

Step-by-Step Operation

Receive the client request

The proxy server listens for client connections on port 8888. When the client requests a file, the proxy accepts the connection and reads the HTTP request.

Example request:

```
GET /test.html HTTP/1.1
```

Host: localhost:8888

1. Extract the requested filename

The proxy examines the request line and extracts the requested path. For the previous request, the extracted path is:

/test.html

2. Determine the cache location

The proxy maps the requested path to a file inside its local cache directory. For example:
/test.html → cache/test.html

3. The proxy creates the cache directory automatically if it does not already exist.

4. Check whether a cached copy exists

The proxy checks whether cache/test.html exists.

Two possible situations can occur:

- The requested file is not in the cache.
- A cached copy of the requested file already exists.

Handle a cache miss

If cache/test.html does not exist, the proxy sends a normal HTTP GET request to the predefined origin server:

```
GET /test.html HTTP/1.1
```

```
Host: 127.0.0.1:9090
```

```
Connection: close
```

5. If the origin server returns 200 OK, the proxy:

- Extracts the response body.
- Saves the file as cache/test.html.
- Sends the downloaded file to the client with a 200 OK response.

6. Responses such as 403 Forbidden or 404 Not Found are forwarded to the client but are not stored in the cache.

Handle an existing cached file

If cache/test.html already exists, the proxy obtains its last-modified time from the file system.

It converts this time into the HTTP date format, such as:

Mon, 03 Aug 2026 21:45:53 GMT

7. Send a conditional request to the origin server

The proxy sends a new request to the origin server containing the cached file's modification date in the If-Modified-Since header:

```
GET /test.html HTTP/1.1 Host: 127.0.0.1:9090
```

```
If-Modified-Since: Mon, 03 Aug 2026 21:45:53 GMT
```

```
Connection: close
```

8. This asks the origin server whether test.html has changed since the cached copy was saved.

Process a 304 Not Modified response

If the origin file has not changed, the origin server responds with:

```
HTTP/1.1 304 Not Modified
```

9. A 304 response does not contain the file body. Therefore, the proxy reads cache/test.html and sends the cached content to the client as a complete 200 OK response.

This avoids downloading the same unchanged file again from the origin server.

10. **Process a 200 OK response**

If the origin file has changed, the origin server responds with 200 OK and includes the updated file.

The proxy then:

- Downloads the updated response body.
- Replaces the old cache/test.html file with the new version.
- Sends the updated file to the client as a 200 OK response.

11. **Close the connections**

After sending the response, the proxy closes the client connection. The origin connection is also closed after its complete response has been received.

How to test the proxy server

Open 3 terminals:

Terminal 1:

```
python3 server.py
```

Terminal 2:

```
python3 proxy_server2.py
```

Terminal 3:

```
curl -i http://localhost:8888/test.html
```

Behavior:

1. First request: origin returns 200; proxy creates cache/test.html.
2. Later requests: proxy sends the cached file's date using If-Modified-Since.
3. Origin returns 304 if unchanged; proxy sends the cached copy as 200 OK.
4. If changed, origin returns 200; proxy updates the cached file.

5. Missing files/Forbidden files are not cached.

```
Mac miniproject % python3 server.py
Serving HTTP on port 9090 ...
+-----+
Established connection with:
IP: 127.0.0.1
Port: 61049
+-----+

Request header:

GET /test.html HTTP/1.1
Host: 127.0.0.1:9090
Connection: close
If-Modified-Since: Mon, 03 Aug 2026 23:39:54 GMT

FILE NOT MODIFIED - SEND 304
[]

Mac miniproject % curl -i http://localhost:8888/test.html
HTTP/1.1 200 OK
Content-Length: 216
Content-Type: text/html; charset=utf-8
Connection: close

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Web Server Test</title>
</head>
<body>
  <h1>200 OK</h1>
  <p>web server successfully found and returned test.html.</p>
  <p>ON PORT 9090</p>
</body>
</html>

Mac miniproject %

Mac miniproject % python3 proxy_server.py
Proxy server listening on port 8888 ...
Predefined origin server: 127.0.0.1:9090
Cache directory: /Users/.../Documents/SFU/CMPT371/MINI_PROJECT/miniproject/cache

[+] Client : ('127.0.0.1', 61048) connected to proxy server...

[+] client request: ( GET /test.html HTTP/1.1
Host: localhost:8888
User-Agent: curl/8.7.1
Accept: */*
)

CACHE EXISTS: /Users/.../Documents/SFU/CMPT371/MINI_PROJECT/miniproject/cache/test.html
If-Modified-Since: Mon, 03 Aug 2026 23:39:54 GMT

Contacting ORIGIN server @: 127.0.0.1 : 9090

ORIGIN request header:
GET /test.html HTTP/1.1
Host: 127.0.0.1:9090
Connection: close
If-Modified-Since: Mon, 03 Aug 2026 23:39:54 GMT

ORIGIN Server response:
b'HTTP/1.1 304 Not Modified\r\nContent-Length: 0\r\nContent-Type: text/plain; charset=utf-8\r\nConnection: close\r\n\r\n'

ORIGIN RESPONSE: 304 Not Modified - sending cached file
[]
```

This caching process reduces unnecessary file transfers while ensuring that the client receives the most recent available version of the requested file.